

Solution Report

Implementation Details

We implemented a **Sequential Hash Join** (Left-Deep Plan) to evaluate the k -line join query. This approach builds the result incrementally by joining the accumulated result with the next relation in the sequence. The code can be found in [Solution.js](#).

Algorithm Steps

The algorithm proceeds in k steps for a query involving k relations.

1. **Step 1:** Join R_1 and R_2 on attribute A_2 .
 - $R_{1-2} = R_1 \bowtie_{A_2} R_2$
 - We used the Hash Join implementation from Problem 1: Hash R_2 on A_2 , then probe with R_1 .
2. **Step 2:** Join the intermediate result R_{1-2} with R_3 on attribute A_3 .
 - $R_{1-3} = R_{1-2} \bowtie_{A_3} R_3$
 - Hash R_3 on A_3 , then probe with R_{1-2} .
3. **Step i :** Join the intermediate result R_{1-i} with R_{i+1} on attribute A_{i+1} .
 - $R_{1-(i+1)} = R_{1-i} \bowtie_{A_{i+1}} R_{i+1}$

This “left-deep” strategy means we always join the current intermediate result (which can be large) with a base relation.

Comparison with Yannakakis

Unlike the Yannakakis algorithm (Problem 2), this approach **does not** guarantee global consistency before joining.

- **Intermediate Results:** We might generate intermediate tuples that are later discarded because they don't match with subsequent relations.
- **Complexity:** In the worst case, the size of intermediate results can be much larger than the input or output, potentially leading to exponential complexity $O(N^k)$ if the join graph is cyclic or if selectivity is poor (though for a line join with simple data, it behaves reasonably). Yannakakis guarantees $O(N + OUT)$.

Execution Results

We ran the sequential algorithm with the same datasets used in Problem 2.

Case 1: k=3

- **Input:** 3 relations (same as Problem 2).
- **Result:** Found **4 tuples**.

(index)	A1	A2	A3	A4
0	1	10	100	1000
1	2	20	200	2000
2	2	20	200	2001
3	4	40	400	4000

Case 2: k=5

- **Input:** 5 relations (chain with noise).
- **Result:** Found **15 tuples**.

(index)	A1	A2	A3	A4	A5	A6
0	0	0	0	0	0	0
1	1000	1000	1000	1000	1000	1000
2	1	1	1	1	1	1
3	2	2	2	2	2	2
4	1002	1002	1002	1002	1002	1002
5	3	3	3	3	3	3
6	4	4	4	4	4	4
7	1004	1004	1004	1004	1004	1004
8	5	5	5	5	5	5
9	6	6	6	6	6	6
10	1006	1006	1006	1006	1006	1006
11	7	7	7	7	7	7
12	8	8	8	8	8	8
13	1008	1008	1008	1008	1008	1008
14	9	9	9	9	9	9

The sequential hash join correctly computes the result, but it constructs full intermediate tuples at each step.